

Autonomous wealth-maximization system architecture ▾

I want you to act as the chief software architect for a brand-new project. Do not base your answer on popular "AI agent" frameworks or multi-agent patterns unless you can justify them from first principles.

First, determine the correct software architecture for the system described below.

Project Goal

Show more

Architected color-coded diagram legend before sequencing remaining deliver...

Aether-class system, first-principles architecture

1. Fundamental computational model

The system is a **persistent stochastic control loop over durable state**. Formally: a controller for a partially observable environment (the economy), where policy = code + accumulated knowledge + stateless judgment calls.

Three axioms drive everything:

1. **Identity is state, not code.** The organism *is* its durable data: beliefs, goals, ledger, memories, decision records. Code and

models are replaceable organs. Kill every process, swap every LLM provider, restart from disk, same organism.

2. **Cognition is a pure function.** (durable state, observation) → (new state, proposed actions). LLMs are stateless evaluators invoked inside this function. They hold no state, own no data, make no final decisions.
3. **Authority must be separated.** Proposing, approving, and executing are different powers. Anything touching money or irreversibility passes a gate that the proposing component cannot bypass.

This rules out chat loops (identity dies with context), workflow engines (task graph must be known upfront, but opportunity discovery is open-ended), and pure reactive scripts (no belief accumulation).

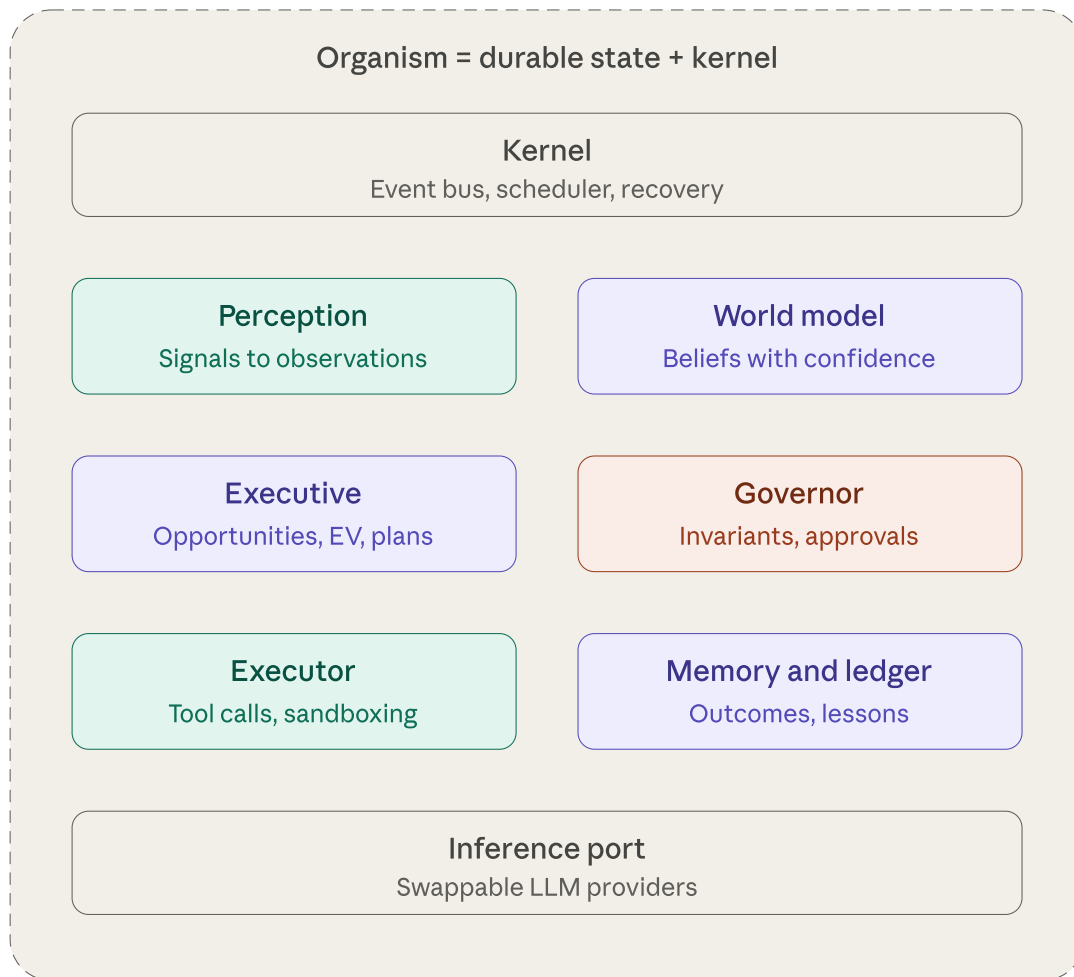
2-4. Core components, responsibilities, data ownership

Seven components. Each owns exactly one kind of data, no shared mutable state.

Component	Responsibility	Owns
Kernel	Event loop, scheduling, timers, crash recovery, lifecycle	Append-only event log, timer table
Perception	Turn external signals into normalized, timestamped observations	Sensor registry, observation log
World model	Convert observations into beliefs with confidence, provenance, decay	Belief store

Component	Responsibility	Owns
Executive	Generate opportunities, estimate expected value, allocate attention and capital, plan	Goal tree, plans, decision records with rationale
Governor	Enforce invariants: budgets, legality rules, approval policy, owner escalation	Constitution, approval log, budget state
Executor	Translate approved plans into tool calls, handle retries and sandboxing	Tool registry, credentials, action log
Memory & ledger	Record outcomes, financial ledger, distilled lessons and playbooks	Episodic memory, ledger, learned heuristics

Plus one piece of infrastructure, not a cognitive component: the **inference port**, a uniform interface any component uses to request judgment from an interchangeable LLM provider. Providers are config, not architecture.



Teal = boundary with the world, purple = cognition and state, coral = authority, gray = infrastructure. External world connects only through Perception (in) and Executor (out). Owner connects only through Governor.

5. Communication

One mechanism: **typed events on the kernel's append-only bus.**
 No component calls another directly, no shared mutable state. Each

component subscribes to event types, reads only its own store, emits new events. Justification:

- Append-only log gives crash recovery and replay for free. Restart = re-read log tail.
- Decoupling: adding a sensor or tool touches nothing else.
- Auditability: every belief, decision, and dirham movement traces to an event chain. Non-negotiable for a system handling money.

Direct calls are allowed for exactly one thing: the inference port, since judgment requests are synchronous and stateless.

6. What persists between cycles

Everything constituting identity: belief store, goal tree, in-flight plans, decision records with rationale, episodic memory, financial ledger, learned heuristics, event log, tool registry, budgets, constitution. What does not persist: LLM contexts, prompts, working computations. If it only lives in a context window, it does not exist.

7. How learning changes behavior

Learning writes durable state consulted at decision time. Never fine-tuning, which would break provider independence. Three loops at three timescales:

1. **Fast, per observation.** World model updates belief confidence, Bayesian-style, with source reliability tracked per sensor.
2. **Medium, per outcome.** Every decision record stored a predicted outcome. Memory scores prediction vs reality, adjusting the Executive's priors on opportunity classes and its calibration (if

EV estimates run 3x optimistic, future estimates get discounted 3x).

3. **Slow, per pattern.** Repeated successes distill into playbooks, reusable procedures the Executive retrieves before planning from scratch. Repeated failures distill into anti-patterns the Governor can enforce.

8. Why this beats common agent architectures

- **Vs chat/single-agent loops:** those conflate cognition with a context window. Identity dies at compaction. Here identity is disk-durable and models are stateless organs.
- **Vs multi-agent (Crew-style roles):** role agents each carry an implicit, divergent world model, coordination via natural-language messages is lossy, and no framework guarantees state durability. Decomposition here follows *data ownership*, not anthropomorphic job titles. If parallelism is later needed, spawn stateless workers under the Executor, that is an implementation detail, not architecture.
- **Vs workflow engines:** they require the task graph upfront. Wealth discovery is open-ended, the graph must be generated at runtime by the Executive.
- **Vs microservices:** seven components in one process with one event log is correct at this scale. Network boundaries add failure modes with zero benefit until components need independent scaling.

9. Trade-offs and failure modes

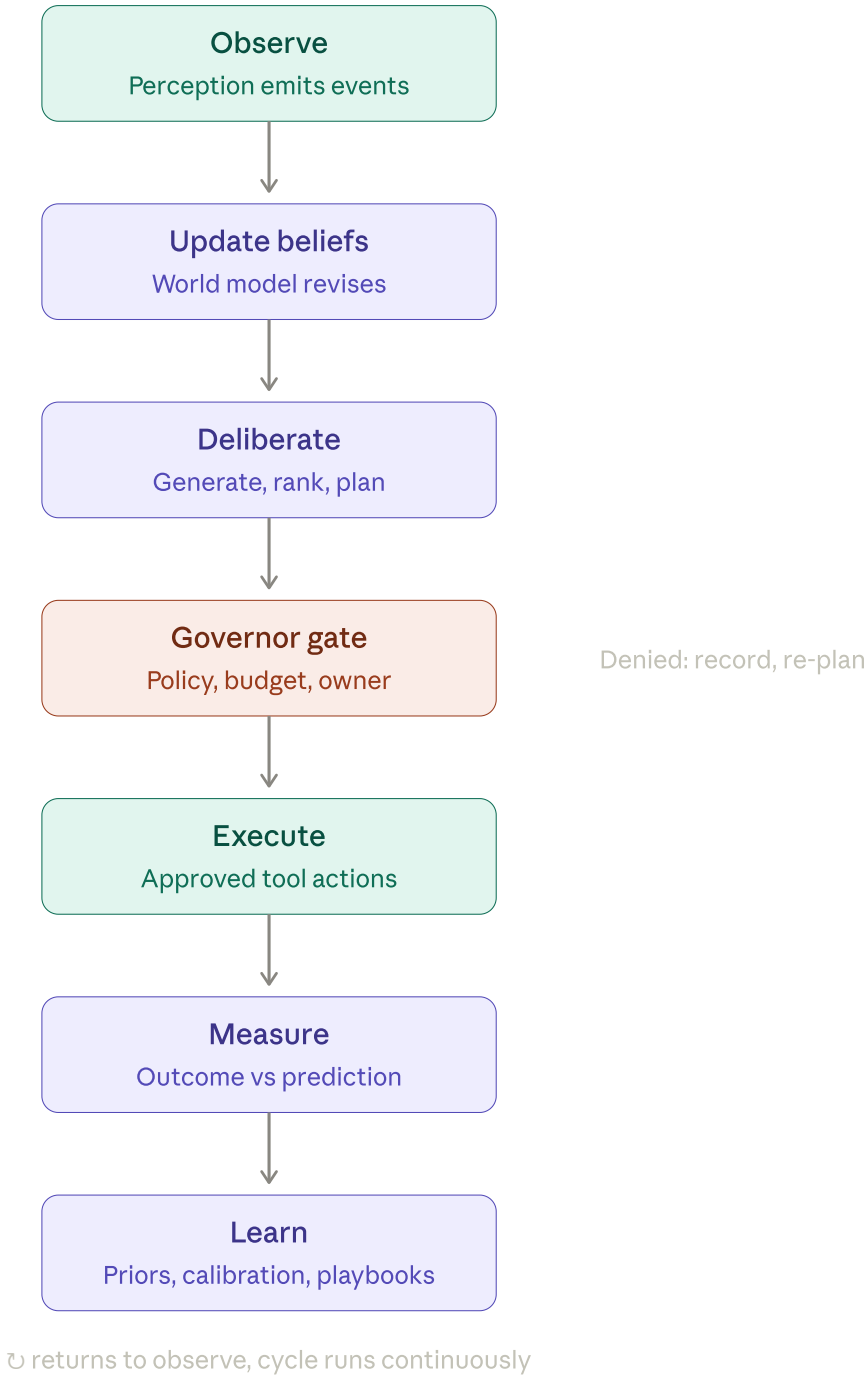
- **Belief corruption compounds.** One bad high-confidence belief poisons downstream decisions. Mitigation: provenance on

every belief, decay, periodic audits.

- **LLM EV estimates are systematically miscalibrated.**
Mitigation: calibration loop above, plus small-bet policy, cap per-opportunity capital until a class proves out.
- **Degenerate opportunity generation.** LLMs converge on the same generic ideas (dropshipping, blog, SaaS). Mitigation: novelty penalty against episodic memory, diverse perception feeds.
- **Kernel is a single point of failure.** Acceptable, it is small and dumb, restart from event log.
- **Ossification vs drift.** A frozen constitution can block adaptation, an editable one invites self-modification exploits. Resolution: constitution amendable only by owner-approved event.
- **Overfitting on tiny n.** Early outcomes are few, learning loop can reinforce luck. Mitigation: minimum sample thresholds before priors shift.
- **Approval bottleneck.** Owner gating irreversible actions caps throughput. Correct trade at this stage, autonomy expands as calibration proves out.
- **Event log growth.** Needs compaction with snapshotting, standard solved problem.

10. Execution flow

One cycle, event-driven, always running:



Two notes on the loop: stages are event-triggered, not lockstep. Perception fires continuously, deliberation fires when beliefs change materially or timers elapse. And a denied plan is data, the denial record feeds the same learning loop as an executed one.

Convergence check: this derivation lands nearly on Aether's frozen Runtime/Perception/Intelligence/Executive/Execution/Memory split, with one structural difference, Governor here is a first-class component with its own store rather than a document plus audit. Worth weighing against the milestone board before treating it as a change, freeze stands otherwise.

